FitBuddy-AI Fitness Plan Generator using Gemini Models

Phase 3 – Project Design

1. Introduction

The design phase transforms the requirements into a technical structure. FitBuddy follows a modular web application design in which the frontend, FastAPI routes, AI modules, database layer, and configuration are separated. This structure makes the application easier to understand, test, maintain, and demonstrate.

2. High-Level Architecture

The main data flow can be represented as: User → HTML Interface → FastAPI Route → Validation/Processing → Gemini AI and Database → Result Page. When a user submits the form, the route receives the form values and creates a validated UserInput object. The workout generator then receives the relevant user information and generates the plan. The nutrition module produces the nutrition tip. Both outputs are stored together with the user record and displayed through the result template.

For feedback, the flow becomes: User Feedback → FastAPI Route → Existing User Lookup → Updated Plan Module → Database Update → Result Page.

3. Application Modules

The main.py module is responsible for creating the FastAPI application, including the router and static-file configuration. The routes.py module contains the application endpoints. It handles the home page, workout generation, feedback submission, admin user viewing, deletion, and health check.

The database.py module contains SQLAlchemy-related database functionality, including the User model, session handling, save, lookup, update, and deletion operations. The schemas.py module defines structured input models such as UserInput and FeedbackRequest.

Inside the AI package, gemini_generator.py is responsible for workout generation, gemini_flash_generator.py is used for nutrition tips, and updated_plan.py handles feedback-based plan revision. The config.py module manages configuration values such as database and administrator settings.

4. Database Design

The User table stores the main information required by the application. The design includes username, user_id, age, weight, goal, intensity, original_plan, updated_plan, feedback, and nutrition_tip. The user_id acts as the identifier used to locate a user's stored record.

When a new user submits the form, a record is created. When an existing user submits the same user ID, the application updates the record instead of creating another entry. This supports the project's requirement for repeated use of the application by the same user.

5. Interface Design

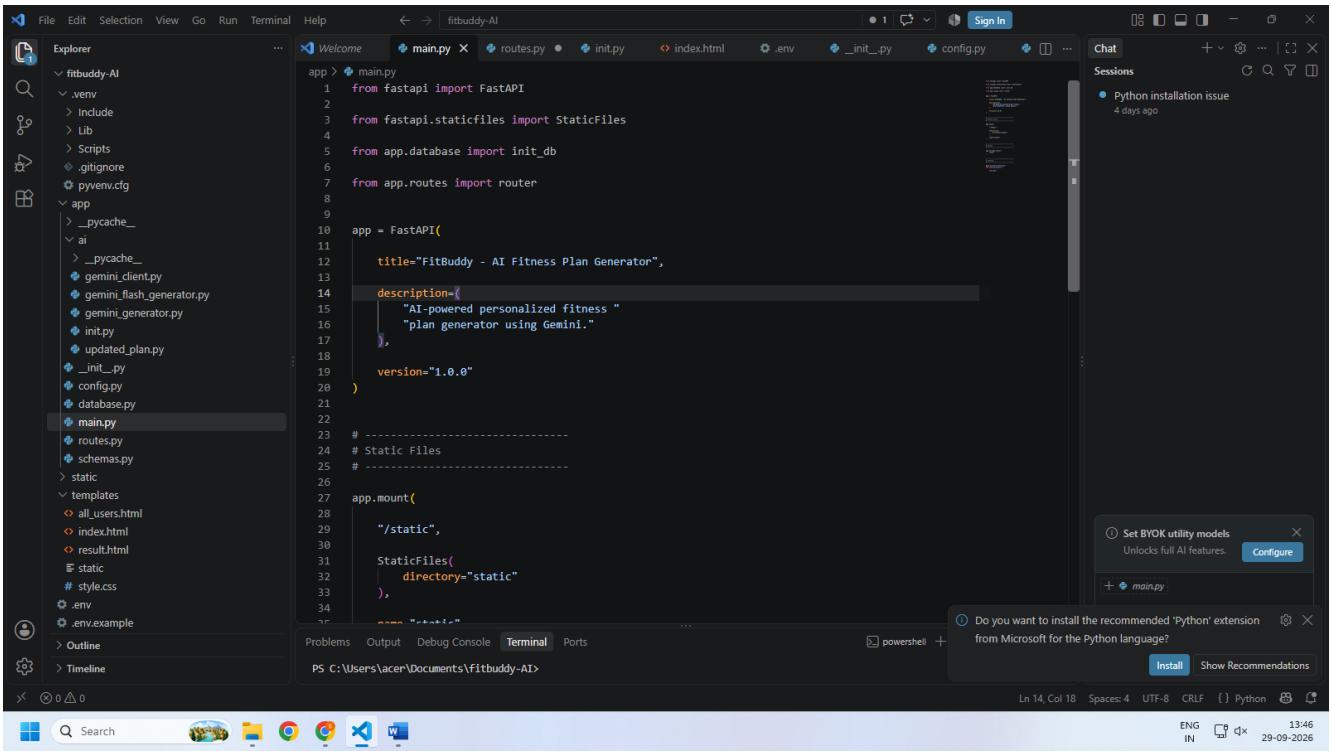
The interface is organized around Jinja2 templates. index.html provides the main input form. result.html displays the generated plan, nutrition information, and feedback-related results. all_users.html is used for the administrative view. CSS resources provide the visual presentation.

The interface design aims to keep the main workflow simple. The user enters details and submits the form. The result is shown on a separate page, where feedback can be provided. This separation makes the workflow easy to demonstrate.

6. Security and Error Handling Design

The administrator functions require an admin key. Invalid keys should result in an access rejection. Form fields are validated before processing. Exceptions during generation or database operations are handled by the route functions so that the user can receive an error response rather than an unhandled application failure.

The design also separates API configuration from the main route logic, allowing sensitive configuration values to be managed outside the main source code when environment variables are used.



7. Conclusion

The design phase establishes how FitBuddy components interact. The modular architecture connects the frontend, FastAPI backend, Gemini AI services, and SQLite database. The design provides the technical blueprint used during the development phase and also gives a clear explanation of the project workflow for documentation and demonstration.

Phase Deliverables / Evidence

Item	Purpose	Status / Evidence
Architecture	Defines component interaction	Documented
Module design	Defines source-file responsibilities	Documented
Database design	Defines stored user data	Documented